

JOBSHEET X

<https://github.com/onlyyourza/PraktikumASD>
STACK

1. Learning Objectives

After completing this session, students will be able to::

1. Create a Stack data structure.
2. Implement Stack algorithms in a Java program.

2. Labs Activities

2.1 Experiment 1: Assignment Submission

Allocated time : 90 Minutes

A number of students submit their assignment files by stacking them on the lecturer's desk, applying the stack principle. The lecturer grades the assignments sequentially, starting from the top file. Refer to the following **Student** Class Diagram.

Student<AttendanceNumber>
nim: String name: String className: String grade: int
Student<AttendanceNumber>() Student<AttendanceNumber>(nim: String, name: String, className: String) grading(grade: int)

Next, to manage the submission of assignment files, a **StudentAssignmentStack** class is needed, serving as a Stack to store student assignment data. The attributes and methods within the **StudentAssignmentStack** class represent data processing using the Stack structure. Refer to the following **StudentAssignmentStack** Class Diagram.

StudentAssignmentStack <AttendanceNumber>
stack: Student[] size: int top: int
StudentAssignmentStack< AttendanceNumber >(size: int) isFull(): boolean isEmpty(): boolean push(mhs): void pop(): Student peek():Student print(): void

Note: The data type of the stack variable should be adjusted according to the type of data to be stored in the Stack. In this exercise, the data to be stored is an array of **Student** objects; therefore, the data type used is **Student**.

2.1.1 Experiment Steps

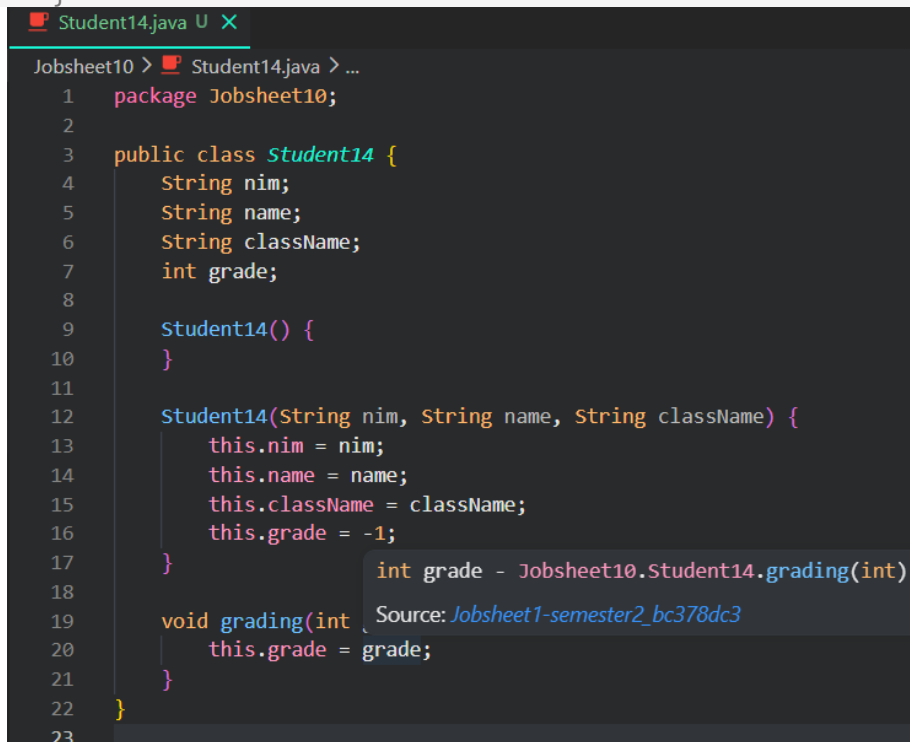
A. Student Class

1. Create a new folder named, for example **Jobsheet10** inside your repository. Then, create a new file and name it **Student<AttendanceNumber>.java**.
2. Complete the **Student** class by adding the attributes shown in the **Student** class diagram, which include **name**, **nim**, **className**, and **grade**.
3. Add a parameterized constructor to the **Student** class as specified in the **Student** class diagram. Set a default value of **grade = -1** to indicate that the assignment has not yet been graded.

```
Student(String nim, String name, String className){  
    this.nim = nim;  
    this.name = name;  
    this.className = className;  
    this.grade = -1;  
}
```

4. Add a **grading()** method to the **Student** class, which is used to set the grade when a student's assignment is evaluated.

```
void grading(int grade){  
    this.grade = grade;  
}
```



```
Student14.java U X  
Jobsheet10 > Student14.java > ...  
1  package Jobsheet10;  
2  
3  public class Student14 {  
4      String nim;  
5      String name;  
6      String className;  
7      int grade;  
8  
9      Student14() {  
10     }  
11  
12     Student14(String nim, String name, String className) {  
13         this.nim = nim;  
14         this.name = name;  
15         this.className = className;  
16         this.grade = -1;  
17     }  
18  
19     void grading(int grade) {  
20         this.grade = grade;  
21     }  
22 }  
23  
int grade - Jobsheet10.Student14.grading(int)  
Source: Jobsheet1-semester2_bc378dc3
```

B. StudentAssignmentStack Class

- After creating the **Student** class, you will then create the **StudentAssignmentStack** <AttendanceNumber>.java class to manage the stack of assignments. The **StudentAssignmentStack** class is an implementation of the Stack data structure.
- Complete the **StudentAssignmentStack** class by adding the attributes shown in the **StudentAssignmentStack** class diagram, which include **stack**, **size**, and **top**.

```
Student[] stack;  
int top, size;
```

- Add a parameterized constructor to the **StudentAssignmentStack** class to initialize the maximum capacity of student assignment data that can be stored in the Stack, as well as to set the initial index of the **top** pointer.

```
StudentAssignmentStack(int size){  
    this.size = size;  
    top = -1;  
    stack = new Student[size];  
}
```

- Next, create a boolean **isFull** method to check whether the stack of student assignments has reached its maximum capacity.

```
boolean isFull(){  
    if(top==size-1){  
        return true;  
    }else{  
        return false;  
    }  
}
```

- Then, create another boolean **isEmpty** method to check whether the stack of assignments is empty.

```
boolean isEmpty(){  
    if(top==-1){  
        return true;  
    }else{  
        return false;  
    }  
}
```

10. To add assignment files to the Stack, create a **push** method. This method should accept a parameter **std**, which is an object of the **Student** class.

```
void push(Student std){
    if(!isFull()){
        top++;
        stack[top] = std;
    }else{
        System.out.println("Stack is already full!!");
    }
}
```

11. The grading of student assignments by the lecturer is done using the **pop** method to remove the assignment to be graded. This method does not accept any parameters but returns an object of the **Student** class.

```
Student pop(){
    if(!isEmpty()){
        Student std = stack[top];
        top--;
        return std;
    }else{
        System.out.println("There is no data in Stack!!");
        return null;
    }
}
```

Note: If information about the student data is required, the return type should be an object of the **Student** class. Conversely, a **void** return type can be used if the student data that is removed will not be processed or used again.

12. Create a **peek** method to check the student assignment at the **top** of the stack.

```
Student peek(){
    if(!isEmpty()){
        return stack[top];
    }else{
        System.out.println("There is no data in Stack!!");
        return null;
    }
}
```

13. Add a **print** method to display the entire list of student assignments in the Stack.

```
void print(){
    for(int i=0;i<=top;i++){
        System.out.println(stack[i].nim+"\t"+stack[i].name+"\t"
            +stack[i].className);
    }
    System.out.println("");
}
```

Teach

```

1  package Jobsheet10;
2
3  public class StudentAssignmentStack14 {
4      Student14[] stack;
5      int top, size;
6
7      StudentAssignmentStack14(int size) {
8          this.size = size;
9          top = -1;
10         stack = new Student14[size];
11     }
12
13     boolean isFull() {
14         if (top == size - 1) {
15             return true;
16         } else {
17             return false;
18         }
19     }
20
21     boolean isEmpty() {
22         if (top == -1) {
23             return true;
24         } else {
25             return false;
26         }
27     }
28
29     void push(Student14 std) {
30         if (!isFull()) {
31             top++;
32             stack[top] = std;
33         } else {
34             System.out.println(x: "Stack is already full!!");
35         }
36     }
37
38     Student14 pop() {
39         if (!isEmpty()) {
40             Student14 std = stack[top];
41             top--;
42             return std;
43         } else {
44             System.out.println(x: "There is no data in Stack!!");
45             return null;
46         }
47     }
48
49     Student14 peek() {
50         if (!isEmpty()) {
51             return stack[top];
52         } else {
53             System.out.println(x: "There is no data in Stack!!");
54             return null;
55         }
56     }
57
58     void print() {
59         for (int i = 0; i <= top; i++) {
60             System.out.println(stack[i].nim + "\t" + stack[i].name + "\t" + stack[i].className);
61         }
62         System.out.println(x: "");
63     }
64 }

```

C. Main Class

14. Create a new main class named **StudentDemo<AttendanceNumber>.java**
15. Add a new **main()** method
16. Within **main()** method, instantiate a new object from **StudentAssignmentStack** class. Give the object name as **stack** and pass **5** as the parameter.

```
StudentAssignmentStack stack = new StudentAssignmentStack(5);
```

17. Declare a Scanner object with the variable name **scan** and an int variable named **choice**.
18. Add a menu to allow users to choose Stack operations for managing student assignment data, using a **do-while** loop structure.

```
do {
    System.out.println("\nMenu:");
    System.out.println("1. Mengumpulkan Tugas");
    System.out.println("2. Menilai Tugas");
    System.out.println("3. Melihat Tugas Teratas");
    System.out.println("4. Melihat Daftar Tugas");
    System.out.print("Pilih: ");
    pilih = scan.nextInt();
    scan.nextLine();
    switch (pilih) {
        case 1:
            System.out.print("Nama: ");
            String nama = scan.nextLine();
            System.out.print("NIM: ");
            String nim = scan.nextLine();
            System.out.print("Kelas: ");
            String kelas = scan.nextLine();
            Mahasiswa mhs = new Mahasiswa(nama, nim, kelas);
            stack.push(mhs);
            System.out.printf("Tugas %s berhasil dikumpulkan\n", mhs.nama);
            break;
        case 2:
            Mahasiswa dinilai = stack.pop();
            if (dinilai != null) {
                System.out.println("Menilai tugas dari " + dinilai.nama);
                System.out.print(s:"Masukkan nilai (0-100): ");
                int nilai = scan.nextInt();
                dinilai.tugasDinilai(nilai);
                System.out.printf(format:"Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
            }
            break;
        case 3:
            Mahasiswa lihat = stack.peek();
            if (lihat != null) {
                System.out.println("Tugas terakhir dikumpulkan oleh " + lihat.nama);
            }
            break;
        case 4:
            System.out.println(x:"Daftar semua tugas");
            System.out.println(x:"Nama\tNIM\tKelas");
            stack.print();
            break;
        default:
            System.out.println(x:"Pilihan tidak valid.");
    }
} while (pilih >= 1 && pilih <= 4);
```



19. Commit and push your code to your Github repository

20. Compile and run your program.

```

Jobsheet10 > StudentDemo14.java > StudentDemo14
1  package Jobsheet10;
2
3  import java.util.Scanner;
4
5  public class StudentDemo14 {
6      Run | Debug
7      public static void main(String[] args) {
8          StudentAssignmentStack14 stack = new StudentAssignmentStack14(size: 5);
9          Scanner scan = new Scanner(System.in);
10         int choice;
11
12         do {
13             System.out.println(x: "\nMenu:");
14             System.out.println(x: "1. Submit Assignment");
15             System.out.println(x: "2. Grade Assignment");
16             System.out.println(x: "3. View Top Assignment");
17             System.out.println(x: "4. View All Assignments");
18             System.out.print(s: "Choose a menu: ");
19             choice = scan.nextInt();
20             scan.nextLine();
21
22             switch (choice) {
23                 case 1:
24                     System.out.print(s: "Name: ");
25                     String name = scan.nextLine();
26                     System.out.print(s: "NIM: ");
27                     String nim = scan.nextLine();
28                     System.out.print(s: "Class Name: ");
29                     String className = scan.nextLine();
30                     Student14 std = new Student14(nim, name, className);
31                     stack.push(std);
32                     System.out.printf(format: "%s's assignment has been successfully submitted!!\n", std.name);
33                     break;
34                 case 2:
35                     Student14 graded = stack.pop();
36                     if (graded != null) {
37                         System.out.println("Grading assignment from " + graded.name);
38
39                         case 2:
40                             Student14 graded = stack.pop();
41                             if (graded != null) {
42                                 System.out.println("Grading assignment from " + graded.name);
43                                 System.out.print(s: "Input grade (0-100): ");
44                                 int grade = scan.nextInt();
45                                 graded.grading(grade);
46                                 System.out.printf(format: "Assignment grade of %s is %d\n", graded.name, grade);
47                             }
48                             break;
49                         case 3:
50                             Student14 latest = stack.peek();
51                             if (latest != null) {
52                                 System.out.println("The last assignment comes from " + latest.name);
53                             }
54                             break;
55                         case 4:
56                             System.out.println(x: "Assignment list:");
57                             System.out.println(x: "Name\tNIM\tClass Name");
58                             stack.print();
59                             break;
60                         default:
61                             System.out.println(x: "Invalid choice.");
62                     }
63             } while (choice >= 1 && choice <= 4);
64
65             scan.close();
66         }
67     }
68 }

```



2.1.2 Verification

Compare the output of your compiled program with the following output.

```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Bima
NIM: 1001
Class Name: 1I
Bima's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Fidela
NIM: 1002
Class Name: 1I
Fidela's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Resty
NIM: 1003
Class Name: 1I
Resty's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 3
The last assignment comes from Resty

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Joseph
NIM: 1004
Class Name: 1I
Joseph's assignment has been successfully submitted!!

Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 4
Assignment list:
Name      NIM      Class Name
1001      Bima     1I
1002      Fidela   1I
1003      Resty    1I
1004      Joseph   1I
```




```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments Choose a menu: 2
Grading assignment from Joseph
Input grade (0-100): 85
Assignment grade of Joseph is 85
```

```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 4
Assignment list:
Name      NIM      Class Name
1001      Bima     1I
1002      Fidela   1I
1003      Resty    1I
```

```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu:
```

```
PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 2\Praktik Algoritma Struktur Data\Jobsheet1-semester2> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\AppData\Roaming\Code\User\workspaceStorage\e0ae0740bf9eef90a1485cc4c4061a4f\redhat.java\jdt_ws\Jobsheet1-semester2_bc378dc3\bin' 'Jobsheet10.StudentDemo14'
```

```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu:
```

2.1.3 Questions

1. Explain the role of the stack data structure in the student assignment management system. Why was a stack used instead of another data structure (e.g., queue or list)?
2. What is the difference between the **push()** and **pop()** methods in a stack, and how are they used in this program?
3. Why is it important to check the condition **!isFull()** before calling the **push()** method? What could go wrong if this check is removed?
4. How many student assignments can be stored in the current implementation of the stack? Provide the specific line of code or variable that determines this.
5. Modify the existing stack implementation so that the system can also show **the first student who submitted their assignment**. Describe the changes you made in both **StudentDemo** and **StudentAssignmentStack** classes.
6. Implement a method to count and return the number of assignments currently stored in the stack. Describe how your method works.
7. What did you learn from this experiment about stack-based systems? Reflect on a real-world application where this kind of system might be useful.
8. Don't forget to synchronize the changes you made from this questions into your repository!

Answer

1. Role of the stack & why not queue/list The stack stores assignment files in LIFO order, exactly like papers physically stacked on the lecturer's desk. The lecturer grades the topmost (most recently submitted) file first. A stack is used because the process is last-in-first-out: the last paper placed on top is graded first. A queue (FIFO) would force the earliest submission to be graded first, and a plain list imposes no clear processing order — neither matches the "grade from the top" rule.
2. Difference between push() and pop() push() inserts a Student14 object on top of the stack and increments top. pop() removes and returns the top object, then decrements top. In this program, push() is used in case 1 (submitting an assignment) and pop() in case 2 (grading = taking the top assignment off the stack to evaluate it).
3. Why check !isFull() before push() isFull() ensures we never write past the last valid index (size - 1). If the check is removed and the stack is already full, top++ makes top = size, so stack[top] accesses stack[size] — an index that doesn't exist — causing an ArrayIndexOutOfBoundsException and crashing the program. The check protects against stack overflow.
4. How many assignments can be stored & the line 5 assignments. Determined by the size argument passed to the constructor in StudentDemo14:

```
StudentAssignmentStack14 stack = new StudentAssignmentStack14(5);
```

5. Show the first student who submitted

```
Student14 peekFirst() {  
    if (!isEmpty()) {  
        return stack[0];  
    } else {  
        System.out.println("There is no data in Stack!!");  
        return null;  
    }  
}
```



I added a `peekFirst()` method in `StudentAssignmentStack14` that returns `stack[0]` (the bottom element = first submitted), and in `StudentDemo14` I added menu option 5 with a case 5 that calls it, plus widened the loop condition to `<= 5`

6. Count the number of assignments in the stack

```
7  ✓   int countAssignments() {  
8      return top + 1;  
9  }
```

My `countAssignments()` method returns `top + 1`. Because `top` is the index of the last element and the array is 0-based, `top + 1` equals the element count. When the stack is empty, `top = -1`, so it correctly returns 0.

7. What I learned & a real-world application I learned that a stack enforces strict LIFO access with constant-time push/pop, making it ideal whenever the most recent item must be handled first. Real-world examples: the browser Back button, undo/redo in text editors, the function call stack, and checking balanced brackets in compilers.



2.2 Experiment 2: Convert Assignment Grade to Binary

Time Allocated: 60 Minutes

In Experiment 2, a new method is added that functions to convert the task value of type **int** into **binary** form after the task has been assigned a value and removed from the Stack.

2.2.1 Experiment Steps

1. Re-open **StudentAssignmentStack<AttendanceNumber>.java**
2. Create a new method named **convertToBinary()** with an **int** parameter **grade**

```
String convertToBinary(int grade){
    ConversionStack stack = new ConversionStack();
    while (grade > 0) {
        int mod = grade % 2;
        stack.push(mod);
        grade = grade / 2;
    }
    String binary = "";
    while (!stack.isEmpty()) {
        binary += stack.pop();
    }
    return binary;
}
```

In this method, the use of **ConversionStack** is implemented, which is similar to the **StudentAssignmentStack** class. The purpose of this is to differentiate the Stack used for students from the Stack used for binary data, as the data types involved are different. Therefore, create a new file named **ConversionStack<AttendanceNumber>.java**.

Note: It is important to remember that all Stack classes essentially have the same operations (methods). What distinguishes them is the specific activities that need to be performed, such as after adding or removing data.

```
public class ConversionStack {
    int[] binaryStack;
    int size;
    int top;

    public ConversionStack() {
        this.size = 32; //asumsi 32 bit
        binaryStack = new int[size];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == size - 1;
    }
}
```

```
public void push(int data) {
    if (isFull()) {
        System.out.println("Stack is already full-filled!!");
    } else {
        top++;
        binaryStack[top] = data;
    }
}

public int pop() {
    if (isEmpty()) {
        System.out.println("Stack is still empty!!");
        return -1;
    } else {
        int data = binaryStack[top];
        top--;
        return data;
    }
}
```

3. To convert the student's assignment value into **binary** form after grading, add a line of code in StudentDemo class, specifically within case 2.

```
case 2:
    Student graded = stack.pop();
    if (graded != null) {
        System.out.println("Grading assignment from " + graded.name);
        System.out.print("Input grade (0-100): ");
        int grade = scan.nextInt();
        graded.grading(grade);
        System.out.printf("Assignment grade of %s is %d\n", graded.name,
grade);
        String binary = stack.convertToBinary(grade);
        System.out.printf("Assignment grade in binary is %s\n", binary);
    }
    break;
```

4. Compile and run the program.
5. **Commit and push the code to your Github repository.**

```

1  package Jobsheet10;
2
3  public class ConversionStack14 {
4      int[] binaryStack;
5      int size;
6      int top;
7
8      public ConversionStack14() {
9          this.size = 32; // asumsi 32 bit
10         binaryStack = new int[size];
11         top = -1;
12     }
13
14     public boolean isEmpty() {
15         return top == -1;
16     }
17
18     public boolean isFull() {
19         return top == size - 1;
20     }
21
22     public void push(int data) {
23         if (isFull()) {
24             System.out.println(x: "Stack is already full-filled!!");
25         } else {
26             top++;
27             binaryStack[top] = data;
28         }
29     }
30
31     public int pop() {
32         if (isEmpty()) {
33             System.out.println(x: "Stack is still empty!!");
34             return -1;
35         } else {
36             int data = binaryStack[top];
37             top--;
38         }
39     }
40
41     public String convertToBinary(int grade) {
42         ConversionStack14 stack = new ConversionStack14();
43         while (grade > 0) {
44             int mod = grade % 2;
45             stack.push(mod);
46             grade = grade / 2;
47         }
48         String binary = "";
49         while (!stack.isEmpty()) {
50             binary += stack.pop();
51         }
52         return binary;
53     }
54
55     public static void main(String[] args) {
56         Scanner scanner = new Scanner(System.in);
57         System.out.print("Format: Assignment grade of %s is %s\n", gradeName, grade);
58         String gradeName = scanner.next();
59         int grade = scanner.nextInt();
60         ConversionStack14 stack = new ConversionStack14();
61         String binary = stack.convertToBinary(grade);
62         System.out.printf("Assignment grade in binary is %s\n", binary);
63     }
64 }

```



2.2.2 Verification

Compare the output of your compiled program with the following output.

```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 1
Name: Excel
NIM: 1001
Class Name: 1I
Excel's assignment has been successfully submitted!!
```

```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
Choose a menu: 2
Grading assignment from Excel
Input grade (0-100): 87
Assignment grade of Excel is 87
Assignment grade in binary is 1010111
```

```
PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 2\Praktik Algoritma Struktur Data\Jobsheet1-semester2> c::; cd 'c:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 2\Praktik Algoritma Struktur Data\Jobsheet1-semester2'; & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\AppData\Roaming\Code\User\workspaceStorage\e0ae0740bf9eef90a1485cc4c4061a4f\redhat.java\jdt_ws\Jobsheet1-semester2_bc378dc3\bin' 'Jobsheet10.StudentDemo14'
```

```
Menu:
1. Submit Assignment
2. Grade Assignment
3. View Top Assignment
4. View All Assignments
5. View First Assignment
6. Count Assignments
Choose a menu: █
```



2.2.3 Questions

1. Explain the workflow of the **convertToBinary()** method.
2. In the **convertToBinary()** method, change the loop condition to **while (grade != 0)**. What is the result? Explain the reason!

Answer

1. Explain the workflow of the **convertToBinary()** method. The method converts a decimal grade into binary using a helper stack. It repeatedly divides the grade by 2; each remainder ($\text{grade} \% 2$, always 0 or 1) is pushed onto a `ConversionStack`. This generates the binary digits in reverse (least-significant digit first). When the loop ends (grade reaches 0), it pops each digit off the stack — the LIFO order reverses them back into the correct sequence (most-significant first) — and concatenates them into the final binary string. Example: 87 pushes remainders 1,1,1,0,1,0,1 (bottom→top), which pop as 1010111.
2. Change the loop condition to **while (grade != 0)** — what is the result? For any valid grade (0–100) the result is exactly the same. Repeatedly applying $\text{grade} = \text{grade} / 2$ on a non-negative number only ever decreases it toward 0 and never makes it negative, so $\text{grade} > 0$ and $\text{grade} != 0$ both become false at the same moment. The two conditions only differ for negative input: $\text{grade} > 0$ would safely return an empty string, while $\text{grade} != 0$ would keep looping and push negative remainders, producing an invalid binary string. Since grades are always ≥ 0 , the output stays identical — > 0 is just the safer guard.



2.4 Assignment

Allocated Time : 90 Minutes

Students submit an excuse letter (due to illness or other reasons) each time they are absent from a lecture. The most recent letter submitted will be processed or validated first by the department administrator. Refer to the class diagram below.

ExcuseLetter<AttendanceNumber>
id: String name: String className: String typeOfExcuse: char duration: int
ExcuseLetter<AttendanceNumber>() ExcuseLetter<AttendanceNumber>(id: String, name: String, className: String, type: char, duration: int)

The **typeOfExcuse** attribute is used to store the type of student leave (S: Sick, or I: Other personal reasons), while the **duration** attribute stores the length of the leave period.

Based on the class diagram, implement a class named **ExcuseLetter**, and add a **ExcuseLetterStack** class to manage the collection of **ExcuseLetter** objects. In the class containing the **main** method, provide the following menu options:

1. **Submit Excuse Letter** – to input new excuse letter data
2. **Process Excuse Letter** – to process or verify the latest excuse letter
3. **View Latest Excuse Letter** – to display the top letter in the stack
4. **Search for Letter** – to search for the presence of a letter by the **student's name**



Jobsheet10 > ExcuseLetter14.java > ExcuseLetter14

```

1  package Jobsheet10;
2
3  public class ExcuseLetter14 {
4      String id;
5      String name;
6      String className;
7      char typeOfExcuse;
8      int duration;
9
10     ExcuseLetter14() {
11     }
12
13     ExcuseLetter14(String id, String name, String className, char type, int duration) {
14         this.id = id;
15         this.name = name;
16         this.className = className;
17         this.typeOfExcuse = type;
18         this.duration = duration;
19     }
20 }
21
22 package Jobsheet10;
23
24 public class ExcuseLetterStack14 {
25     ExcuseLetter14[] stack;
26     int top, size;
27
28     ExcuseLetterStack14(int size) {
29         this.size = size;
30         top = -1;
31         stack = new ExcuseLetter14[size];
32     }
33
34     boolean isFull() {
35         return top == size - 1;
36     }
37
38     boolean isEmpty() {
39         return top == -1;
40     }
41
42     void push(ExcuseLetter14 letter) {
43         if (!isFull()) {
44             top++;
45             stack[top] = letter;
46         } else {
47             System.out.println(x: "Stack is already full!!");
48         }
49     }
50
51     ExcuseLetter14 pop() {
52         if (!isEmpty()) {
53             ExcuseLetter14 letter = stack[top];
54             top--;
55             return letter;
56         } else {
57             System.out.println(x: "There is no excuse letter in the stack!!");
58             return null;
59         }
60     }
61 }

```

```
30  ExcuseLetter14 pop() {
31      if (!isEmpty()) {
32          ExcuseLetter14 letter = stack[top];
33          top--;
34          return letter;
35      } else {
36          System.out.println(x: "There is no excuse letter in the stack!!");
37          return null;
38      }
39  }
40
41  ExcuseLetter14 peek() {
42      if (!isEmpty()) {
43          return stack[top];
44      } else {
45          System.out.println(x: "There is no excuse letter in the stack!!");
46          return null;
47      }
48  }
49
50  ExcuseLetter14 search(String name) {
51      for (int i = top; i >= 0; i--) {
52          if (stack[i].name.equalsIgnoreCase(name)) {
53              return stack[i];
54          }
55      }
56      return null;
57  }
58  }
59
```



Jobsheet10 > ExcuseLetterDemo14.java > ExcuseLetterDemo14

```
1  package Jobsheet10;
2
3  import java.util.Scanner;
4
5  public class ExcuseLetterDemo14 {
    Run | Debug
6      public static void main(String[] args) {
7          ExcuseLetterStack14 stack = new ExcuseLetterStack14(size: 10);
8          Scanner scan = new Scanner(System.in);
9          int choice;
10
11         do {
12             System.out.println(x: "\nMenu:");
13             System.out.println(x: "1. Submit Excuse Letter");
14             System.out.println(x: "2. Process Excuse Letter");
15             System.out.println(x: "3. View Latest Excuse Letter");
16             System.out.println(x: "4. Search for Letter");
17             System.out.print(s: "Choose a menu: ");
18             choice = scan.nextInt();
19             scan.nextLine();
20
21             switch (choice) {
22                 case 1:
23                     System.out.print(s: "ID: ");
24                     String id = scan.nextLine();
25                     System.out.print(s: "Name: ");
26                     String name = scan.nextLine();
27                     System.out.print(s: "Class Name: ");
28                     String className = scan.nextLine();
29                     System.out.print(s: "Type of Excuse (S = Sick, I = Other): ");
30                     char type = scan.nextLine().charAt(index: 0);
31                     System.out.print(s: "Duration (days): ");
32                     int duration = scan.nextInt();
33                     scan.nextLine();
34                     ExcuseLetter14 letter = new ExcuseLetter14(id, name, className, type, duration);
35                     stack.push(letter);
36                     System.out.printf(format: "Excuse letter from %s has been submitted!!\n", letter.name);
```



```

5 public class ExcuseLetterDemo14 {
6     public static void main(String[] args) {
34         ExcuseLetter14 letter = new ExcuseLetter14(id, name, className, type, duration);
35         stack.push(letter);
36         System.out.printf(format: "Excuse letter from %s has been submitted!!\n", letter.name);
37         break;
38     case 2:
39         ExcuseLetter14 processed = stack.pop();
40         if (processed != null) {
41             System.out.println("Processing excuse letter from " + processed.name
42                 + " (" + describeType(processed.typeOfExcuse)
43                 + ", " + processed.duration + " day(s))");
44         }
45         break;
46     case 3:
47         ExcuseLetter14 latest = stack.peek();
48         if (latest != null) {
49             System.out.println("Latest excuse letter is from " + latest.name
50                 + " (" + describeType(latest.typeOfExcuse)
51                 + ", " + latest.duration + " day(s))");
52         }
53         break;
54     case 4:
55         System.out.print(s: "Enter the student's name to search: ");
56         String keyword = scan.nextLine();
57         ExcuseLetter14 found = stack.search(keyword);
58         if (found != null) {
59             System.out.println("Letter found! " + found.name + " - "
60                 + describeType(found.typeOfExcuse) + ", "
61                 + found.duration + " day(s), class " + found.className);
62         } else {
63             System.out.println("No excuse letter found under the name " + keyword);
64         }
65         break;
66     default:
67         System.out.println(x: "Invalid choice.");
68     }
}

```

```
54         case 4:
55             System.out.print(s: "Enter the student's name to search: ");
56             String keyword = scan.nextLine();
57             ExcuseLetter14 found = stack.search(keyword);
58             if (found != null) {
59                 System.out.println("Letter found! " + found.name + " - "
60                                     + describeType(found.typeOfExcuse) + ", "
61                                     + found.duration + " day(s), class " + found.className);
62             } else {
63                 System.out.println("No excuse letter found under the name " + keyword);
64             }
65             break;
66         default:
67             System.out.println(x: "Invalid choice.");
68     }
69     while (choice >= 1 && choice <= 4);
70
71     scan.close();
72 }
73
74 static String describeType(char type) {
75     if (type == 'S' || type == 's') {
76         return "Sick";
77     } else if (type == 'I' || type == 'i') {
78         return "Other personal reason";
79     } else {
80         return "Unknown";
81     }
82 }
83 }
84
```